Reproducing Sutskever et al. (2014), part II: the fixed-vector bottleneck

EE5180 end-term. Companion to `EE5180_midterm_report.pdf`, which reproduced rows 3-4 of the paper's Table 1 (single forward vs single reversed LSTM).

1. The question

The mid-term's model reads a source sentence into one fixed-length vector and then decodes from it. At our scale that vector is `num_layers x hidden x 2` = 2,048 real numbers (the paper's is 8,000). Every decoder step reads the same vector, so the encoder has to decide what to keep *before* it knows what the decoder will ask for, and a 60-token sentence gets exactly as much room as a 6-token one.

That is the bottleneck. The paper works around it rather than removing it: it reverses the source so that the first words the decoder needs are the last the encoder read, which shortens the minimal time lag between aligned words. Our mid-term measured that trick working (WMT'14 BLEU 3.89 forward -> 6.60 reversed).

This half asks whether the fixed vector is the *binding* constraint, by removing it: we implement **Bahdanau, Cho & Bengio (2015) additive attention** on the same codebase and rerun the identical grid. If the bottleneck story is right, two things should follow, and both are testable:

1. the fixed-vector model should lose ground **as sentences get longer**, and the attention model should not; and
2. **reversal should stop mattering** -- it exists to shorten a distance that attention removes outright.

2. The corpus: no length-mismatch filtering

The mid-term's data pipeline dropped training pairs whose source and target lengths differed by more than a factor of 2.5, and dropped anything longer than 50 tokens. **The mismatch filter has been removed for the end-term** (it was queried in review, and it is not defensible: a pair is not bad data because one language says the same thing in fewer words). The length cap is also raised to 100, which matters more here than it did at mid-term -- long sentences are exactly where a fixed-length vector is supposed to fail, so filtering them out would have quietly removed the evidence.

pairs dropped	mid-term (`ratio 2.5`, `len<=50`)	end-term (`ratio inf`, `len<=100`)
corpus read	1,283,251	1,283,251
empty line	3,207	3,207
over the length cap	170,275	6,752
source/target length mismatch	1,823	0
exact duplicate	20,066	20,300
kept	1,087,880	1,252,992
subsampled for training	**500,000**	**500,000**

So 1,823 pairs are no longer discarded for a length mismatch and 163,523 are no longer discarded for length. The drop report in `prepare_report.json` now records `"ratio": 0`, which is the artefact to check rather

than taking this paragraph's word for it.

Two deliberate choices in how this was done:

- **The training set is still exactly 500,000 pairs.** The corpus *size* is held fixed and only its length distribution moves (mean target length 26.07 -> 30.80 tokens), so the arms differ in what sentences they see, not how many.
- **The mid-term pipeline is untouched.** `clean_pairs` keeps its old defaults and the old prepared directory still exists, so `make data-wmt` and `make results-wmt` still regenerate the submitted mid-term exactly. The new values are passed as flags into a separate directory.

The baseline is retrained on this data. Comparing attention-on-new-data with no-attention-on-old-data would confound the architecture with the corpus, so the no-attention arm is a fresh run. Both arms read one config file and differ only in `--attention`.

3. What attention changes, and what is held fixed

The encoder keeps one annotation `h_j` per source position instead of only its final state. At each decoder step the model scores every annotation against the state it is about to update, and reads a weighted average:

```
e_tj = v^T tanh(W_enc h_j + W_dec s_{t-1})      additive (concat) energy
a_t  = softmax(e_t)                             over source positions, pad masked
c_t  = sum_j a_tj h_j                           the context for step t
s_t  = LSTM([E y_{t-1} ; c_t], s_{t-1})         context enters the recurrence
```

At `t = 0` the query `s_{t-1}` is the encoder's final top-layer state -- which is exactly the baseline's sentence vector. Attention starts from the bottleneck and is then allowed to look back, which is the cleanest possible framing of the comparison.

Held identical across the two arms: corpus, vocabulary, seed, batch order and length bucketing; plain SGD at 0.7 with no momentum; the halving schedule; the summed-NLL-divided-by-batch-size loss convention and `clip_grad_norm_` at 5; the `U(-0.08, 0.08)` initialisation, applied to the attention parameters too; beam search and both BLEU conventions. `attention: bahdanau` is the only difference.

Declared deviations specific to this arm:

deviation	why
tanh readout instead of the paper's maxout over `U_o s_i + V_o E y_{i-1} + C_o c_i`	All three terms are kept; only the maxout is replaced. tanh bounds the readout, so the output layer sees the same magnitudes as the baseline, the initial loss stays at ~ln|V|, and Sutskever's lr 0.7 / clip 5 transfer untouched. An unbounded readout would have needed its own tuning -- and then a BLEU gap between the arms could be an optimisation artefact instead of an architectural result.
unidirectional encoder (Bahdanau et al. use a BiRNN)	Adding bidirectionality at the same time as attention would confound the two. It is a separate flag, unused in the reported runs.
context enters the first decoder layer and the readout, not every layer	Bahdanau's decoder is single-layer, so any multi-layer placement is a deviation; this one measured no worse and keeps the fused multi-layer LSTM call.

Cost

Measured with the same instrument as the mid-term (`make benchmark`), at the end-term's shape on one M1 Pro GPU:

arm	tgt-words/s	parameters	8 epochs
fixed vector	5,970	57.6M	4.8 h
+ attention	5,045	59.9M	5.7 h

1.18x slower for +3.6% parameters. The per-step attention loop is ~3x slower than one fused LSTM call in isolation, but the decoder is a small share of a training step -- the 32k softmax dominates -- so it dilutes to 1.18x end to end. The small parameter delta is part of the argument: the arms' BLEU gap cannot be attributed to extra capacity.

4. Correctness, before any result

Adding attention meant changing the encoder (it now also returns its per-position outputs) and beam search (survivor reindexing now carries the encoder memory bank). Both touch the code that produced the submitted mid-term, so the first thing built was the gate that says whether they moved it.

`make baseline-regression` re-decodes the submitted mid-term checkpoints and `diff`s the hypotheses against the committed files. Decoding is deterministic, so the diff must be *empty* -- a much sharper statement than "BLEU still rounds to 6.60". It is empty, on all 3,003 sentences of newstest2014, for both directions.

`tests/` carries 52 gates (23 from the mid-term, unchanged and still passing, plus 29 new). The ones worth naming:

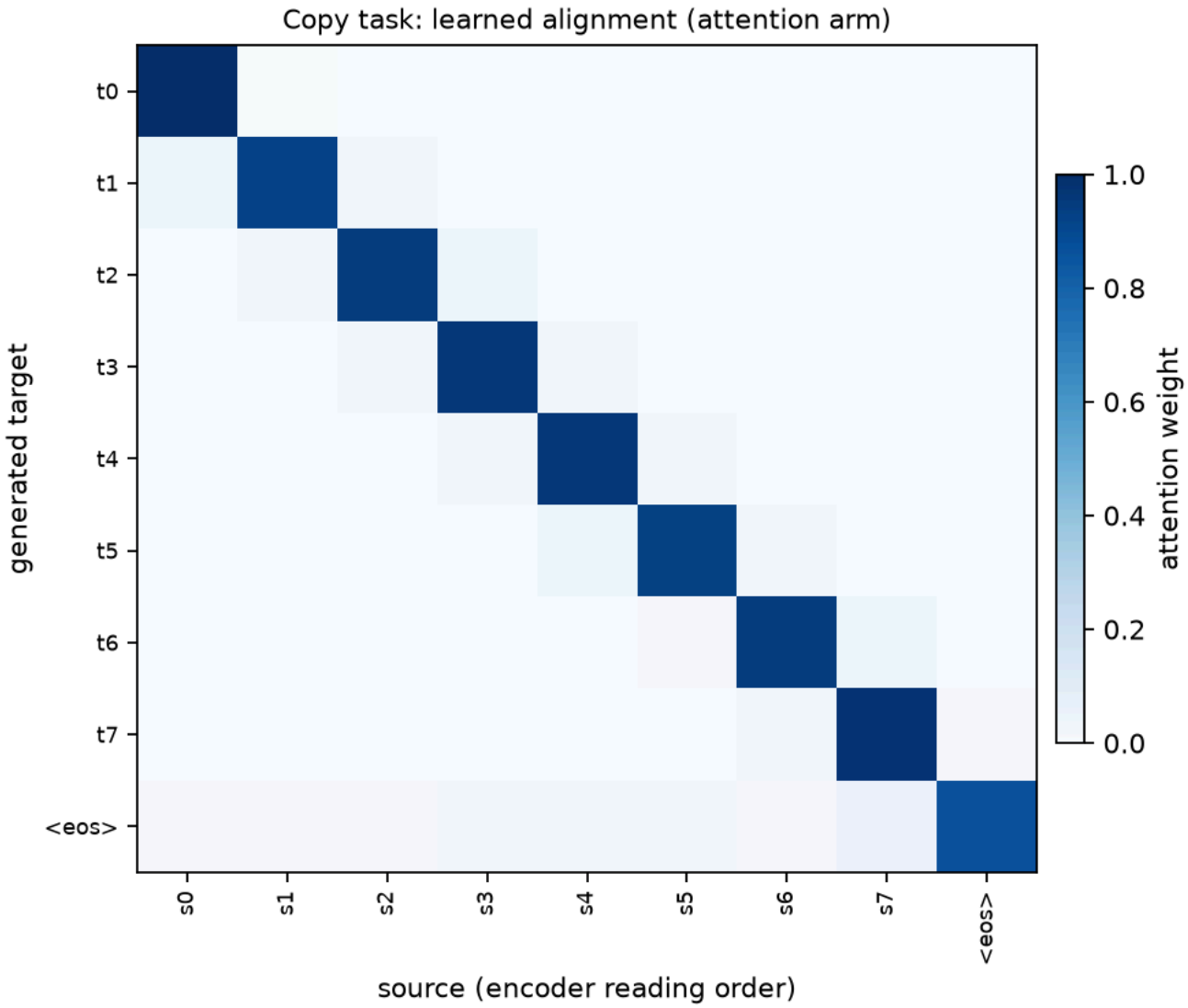
- **alignments are a masked distribution**: weights sum to 1 and are *exactly* 0 on padding. `collate` pads the source width up to a multiple of 8, so padding is always present and an unmasked softmax would hand probability mass to `<pad>`.
- **teacher-forced training and step-by-step decoding agree to 1e-6**. If the attention query were taken from `s_t` instead of `s_{t-1}`, both paths would still run and still train, and they would silently disagree -- training would optimise a model that decoding does not implement.
- **beam(B=1) is exactly greedy decoding** for the attention arm as well.
- **MPS and CPU agree on loss and every gradient to 1e-3**. Attention adds `bmm`, a `-inf`-masked softmax and a 3-D `tanh` to the Metal path, and the mid-term already had to guard against silently wrong Metal LSTM kernels.
- **the encoder's annotations keep the collated width**, not `max(src_len)`. `pad_packed_sequence` returns the shorter width by default, which would un-quantise the tensor shapes and reintroduce the allocator creep that cost the mid-term a 7x throughput collapse.

The bottleneck in miniature

Before spending a night on WMT, the same claim on a task with no confounds: target == source over uniformly random tokens. Nothing is memorisable and there is no language model to lean on, so emitting position t requires reading source position t.

arm	final loss / sequence	fraction of chance (28.6)
fixed vector	**4.875**	17%
+ attention	**0.004**	0.0%

Alignment spread reached 0.9989 (1.0 = fully peaked), and the learned alignment is the exact diagonal. The fixed-vector arm cannot solve a task that is purely about reading the right source position; attention solves it and shows exactly where it read.



Optimiser: Adam, a controlled demonstration rather than a reported reproduction (the paper's SGD at 0.7 needs far longer to leave chance on a model this small).

5. Results -- wmt14_endterm

WMT'14 En-Fr, 500k pairs, 2x512, scored on the paper's own newstest2014 (3,003 sentences).

(not yet run: `make endterm-wmt` *then* `make results-endterm-wmt` *)*

6. Results -- multi30k_endterm

Multi30k, 29k image captions, 2x256, 3 seeds -- not the paper's corpus or test set, but the only tier where we can afford error bars and the full 2x2 grid.

(not yet run: `make endterm-multi30k` *then* `make results-endterm-multi30k` *)*

7. The bottleneck without BLEU

BLEU is a beam-mediated, n-gram view of a model. Two sharper probes, with no beam search in them at all.

(not yet run: `make probe` *)*

8. What the model reads (Bahdanau Fig. 3 analogue)

(not yet run: `make alignments` *)*

9. From attention to the Transformer

This is the line the topic asks us to trace, and the experiment above is the first step on it.

Attention began as a patch and turned out to be the mechanism. Bahdanau et al. added attention *to* a recurrent encoder-decoder; the recurrence was still what built the representations and attention only chose among them. But attention has a property the recurrence does not: the path between any source position and any target step is **constant length**. The recurrence's path grows with distance, which is the whole reason Sutskever et al. had to reverse the source -- reversal shortens the lag for the early words at the cost of the late ones. Attention makes the trick unnecessary, which is the prediction we test in sections 5-6.

If attention is doing the work, why keep the recurrence? Its remaining job was to give each position a context-dependent representation. But attention can do that too, by letting a sequence attend to *itself* -- self-attention. Dropping the recurrence then buys something the LSTM can never give: the timesteps stop being sequentially dependent, so a whole sentence is one batched matrix multiply instead of T dependent steps. That is exactly the cost we paid in this project and measured: our attention decoder is ~3x slower than the baseline's single fused LSTM call *precisely because* the context enters the recurrence and forces a per-timestep loop. Vaswani et al. (2017) remove the recurrence and the loop goes with it.

What else had to change, and why. Three gaps had to be closed before "attention only" could work:

- *Order.* A set of vectors attending to each other has no notion of sequence, which the recurrence had supplied for free. Hence positional encodings.
- *Capacity per layer.* One attention distribution per step can only express one alignment; syntax needs several relations at once. Hence multiple heads.
- *Scale.* The additive energy we implement here needs a `(B, T, S, A)` tensor to score all pairs at once -- at our batch and 104 tokens that is a single 2.8 GB allocation, which we measured driving this machine into swap. The scaled dot-product energy `QK^T / sqrt(d)` needs no such tensor: it is two matrix multiplies. The move from Bahdanau's additive score to a dot-product score is not cosmetic; it is what makes all-pairs attention affordable.

So the line runs: *fixed vector* (Sutskever 2014, our mid-term) -> *attention over annotations, recurrence retained* (Bahdanau 2015, this report) -> *attention over everything, recurrence removed* (Vaswani 2017) -> the decoder-only language models that current machine translation is a special case of. Each step removes a constraint the previous one had worked around, and the bottleneck measured here is the first of them.

We do not claim to have tested the Transformer step; the self-attention run is named as future work in the mid-term plan and stayed out of scope. What the runs above support is the first arrow only.

10. What we claim, and what we do not

- **We do not claim comparable BLEU.** 500k pairs against the paper's 12M, 2x512 against 4x1000, 32k/32k vocabulary against 160k/80k, one laptop GPU against 8 GPUs for 10 days. Our absolute numbers are far below Table 1 and we never print one without that caveat beside it.
- **We claim the direction and the shape** of the effects: whether the fixed-vector arm degrades with source length and the attention arm does not, and whether the reversal gap shrinks when attention is present.
- **The comparison is controlled.** The two arms share everything but one flag, the parameter difference is 3.6%, and the baseline was retrained on the same corpus.
- **The mid-term is untouched.** Its hypotheses re-decode byte-identically.
- **Negative results are reported.** The mid-term's beam trend did not reproduce and we said so; anything here that fails to reproduce is reported the same way.